

Technical Datasheet 4G Gateway

Product Overview:-

The 4G Gateway is an advanced networking device designed to provide seamless connectivity and data management capabilities for various applications. With support for 4G LTE connectivity and TCP Ethernet, cloud connectivity via MQTT protocol, and robust storage options, it offers a comprehensive solution for remote monitoring and control systems. Equipped with Modbus RS485 port and micro-Sim compatibility, it caters to a wide range of industrial and IoT applications. The intuitive web-based UI interface allows for easy configuration and management, making it an ideal choice for both professionals and enthusiasts.



Specifications

General:-

- **4G LTE Connectivity:** Utilizes 4G LTE technology for reliable and high-speed internet access. If both 4G, Wi-Fi and Ethernet are unavailable, the data is securely stored in the SD card for later upload.
- **Cloud Connectivity (MQTT):** Seamlessly connects to cloud platforms using the MQTT protocol for efficient data transmission and management.
- **Event Storage:** Offers on board event storage, capable of storing up to 50,000 events or more. Includes an external memory card slot for easy data retrieval in the event of 4G network issues.
- **Modbus RS485 Port:** Features 1 port of Modbus RS485 for integration with industrial control systems and sensors.
- **Micro Sim Compatibility:** Supports micro-Sim card for flexible and convenient connectivity option.
- **Web-based UI Interface:** Intuitive web-based user interface for easy configuration, monitoring, and management.
- **Dimensions:** 110 mm L x 110 mm B x 50 mm H.
- **Power:** Input Power – 15-40 VDC, Typical – 12V DC @ 150mA.
- **Operating Temperature:** 0 – 60C (32 ~ 140F).
- **Storage Temperature:** 20 - 70C (-4 ~ 158F).
- **Storage Humidity:** 5 ~ 95 % RH, non – Condensing.

LED Indicators:-

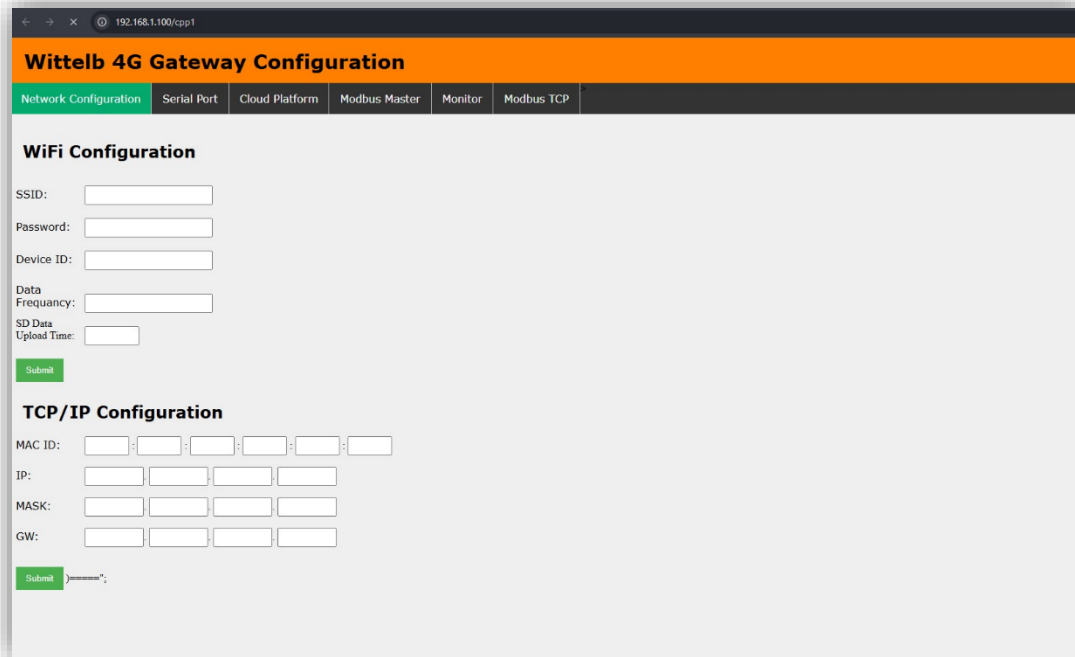
- **POWER:** Device power status.
- **NETWORK:** Indicates 4G/Wi-Fi network connectivity.
- **CLOUD:** Shows MQTT cloud connection status.
- **POLL:** POLL/WI-FI: Indicates Modbus polling activity / Wi-Fi mode
- **RX:** Data received from RS485.
- **TX:** Data transmitted to RS485.



Configuration Settings:

- Power up the module by connecting a 12 V DC supply.
- Insert the SIM card into the designated slot.
- Place the Micro SD card into its designated slot.
- Connect MODBUS RTU Multiple Slave devices (up to 31) to the A & B Terminal Of Module.
- Access the module's configuration hotspot (ID: WIN-4G-GATEWAY, PASS: 123456789).
- Default IP Address of Module is "192.168.1.100/cpp1".
- Upon entering the provided IP address, a configuration webpage Pages will be accessible.

Page 1: -



Wittelb 4G Gateway Configuration

Network Configuration | Serial Port | Cloud Platform | Modbus Master | Monitor | Modbus TCP

WiFi Configuration

SSID:

Password:

Device ID:

Data Frequency:

SD Data Upload Time:

TCP/IP Configuration

MAC ID: : : : : :

IP: . . .

MASK: . . .

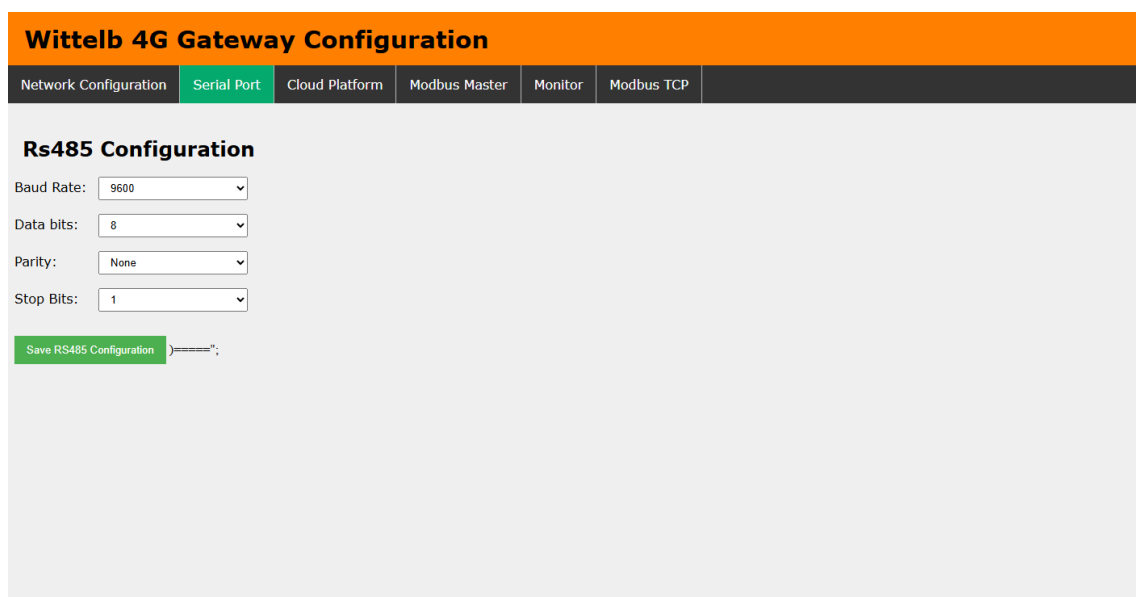
GW: . . .

- Enter Device ID (Should be unique for every device).
- Enter data frequency in min to receive data on cloud.
- Enter time to upload SD card data to cloud.

Page 2:-

Our focus is You

On second page you can set device Baud Rate (9600 / 19200 / 38400 / 57600 / 115200), Data Bits (5 / 6 / 7 / 8), Parity (None / Odd / Even), and Stop Bits (1 / 2) for RS485 Communication



Wittelb 4G Gateway Configuration

Network Configuration | Serial Port | Cloud Platform | Modbus Master | Monitor | Modbus TCP

Rs485 Configuration

Baud Rate:

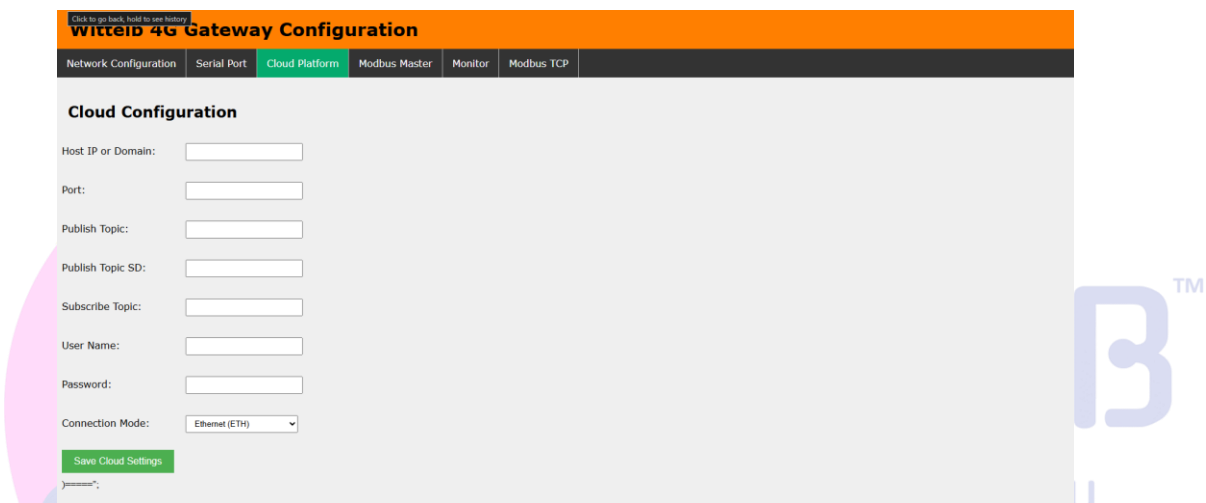
Data bits:

Parity:

Stop Bits:

Page 3:-

- Navigate to page 3 on the cloud platform.
- Input the Host IP or Domain.
- Enter the port Number.
- Enter the topics for publishing and receiving messages.
- Provide a username and password if required.
- Allow a few seconds for the module to restart and establish a connection to MQTT.
- Select Wi-fi, TCP and 4G from Which Source you need to send data over MQTT



Page 4:-

- Go to page 4 on the Modbus Master interface.
- Input the name and slave address for the designated slave.
- Specify the register address from which you want to read slave data.
- You can enter only the Register which you need to read.
- Select the Data Type: UINT16, Float32 Big Endian, Float32 Little Endian, Float32 BE Byte Swap, or Float32 LE Byte Swap.
- Enter the function code corresponding to the desired slave.

Wittelb 4G Gateway Configuration

Network Configuration
 Serial Port
 Cloud Platform
 Modbus Master
 Monitor
 Modbus TCP
 About

Modbus Master Configuration

Name	Slave Address	Register	Function Code	Data Type	Actions
Slave	1	12	3	UNIT16 UNIT16 Float32 Big Endian Float32 Little Endian Float32 BE Byte Swap Float32 LE Byte Swap	Delete

Page5:-

- On page 5 (Monitor), view live Modbus register values for all configured entries.
- Table auto-refreshes every 7 seconds showing: Slave Address, Register, Function Code, Tag Name, Data Type, Live Value, and Status (OK / FAIL)
- Use Slave Address and Register filter fields to narrow down the live view.

Wittelb 4G Gateway Configuration

Network Configuration
 Serial Port
 Cloud Platform
 Modbus Master
 Monitor
 Modbus TCP
 About

Modbus Live Monitor

Total Registers

Slaves Active

Showing

Slave Address

e.g. 1

Register

e.g. 100

Connection failed — retrying...

Next refresh in 3s

Sr.No	Slave Address	Register	Function Code	Name / Tag	Data Type	Live Value	Status	Last Updated
Loading data...								

Modbus Slave Configuration:

The gateway supports reading up to 100 individual Modbus points (e.g., sensor values, coil states, pump statuses), stored as entries indexed from 0 to 99. Each entry requires five specific parameters to successfully poll the connected RS485 field devices.

Configuration Parameters:

- **Slave Index (0-99):** The internal row position for storage on the gateway, which is not transmitted over the Modbus line.
- **Slave Name:** A custom alphanumeric label (up to 19 characters) used as the JSON key during MQTT transmission. If left blank, the device auto-generates a name based on the address (e.g., S1_REG40001).
- **Slave ID:** The hardware RS485 address of the target device, which must be configured between 1 and 31. Multiple entries can share the same Slave ID.
- **Function Code:** Defines the Modbus reading method. The gateway currently supports Code 1 (Coil Read for ON/OFF status), Code 3 (Holding Register Read for settings/data), and Code 4 (Input Register Read for measured values).
- **Data Type:** Defines how the raw register data is parsed (applicable only to Function Codes 3 and 4).
 - **Type 0:** Simple Integer (Reads 1 Register).
 - **Type 1-4:** Float / Decimal Number (Reads 2 Registers). The 1 through 4 selection alters the byte order; if a float value reads incorrectly, rotating through orders 1-4 typically resolves it. Data type is irrelevant for Coil reads, which always yield 0 or 1.

Register Addressing & Offset Logic:

Modbus datasheet addresses (e.g., 40001) are not transmitted directly to the slave device. The first digit indicates the register type (0 = Coil, 3 = Input, 4 = Holding) and is discarded because the Function Code dictates the type.

Standard Modbus protocol requires subtracting 1 from the remaining digits. Because some manufacturers pre-subtract this offset in their documentation, adding or subtracting 1 is the primary troubleshooting step if a register fails to read correctly.

Datasheet Register	Type Prefix Removed	Final Entered Value (Subtract 1)
40001 (Holding)	0001	0
40010 (Holding)	0010	9
30005 (Input)	0005	4
00001 (Coil)	0001	0

MQTT Communication & JSON Payloads:

The gateway acts as an MQTT client, grouping data per Modbus slave address into a single JSON object and transmitting it over the network.

Telemetry Publishing (Device -> Cloud):

- **Primary Topic:** Data is published to `publish_topic`, defined in the web-config panel.
- **Payload Structure:** Each JSON payload includes the Device ID, a Unix timestamp (sent as a string), the integer Slave Address, and the grouped register values using the configured Slave Names.
- **Data Formatting:** Coil reads produce 0/1 integers, while 16-bit register reads yield plain integers. 32-bit float reads are automatically rounded to 4 decimal places. If a float registers as NaN or infinite, it is transmitted as a null value rather than a number.
- **SD-Card Fallback:** If the MQTT connection drops, the JSON payload is saved to the local SD card. Once connectivity is restored, the buffered data is republished unchanged to a secondary topic (`pub_topic1`).

Command Reception (Cloud -> Device):

- **Target Topic:** The gateway listens for remote write commands on `subscriber_topic`.
- **Payload Requirements:** To trigger a write action, the incoming JSON must contain the target slave Address, the specific register, and the value to be written.
- **Execution Limitation:** Currently, the MQTT command engine strictly enforces Modbus Function Code 6 (Write Single Register) for all incoming writes. While the internal firmware supports Coils (FC5) and multi-registers (FC15/FC16), these are not yet mapped to the MQTT listener.

The following table outlines the JSON payloads handled by the MQTT engine, their communication direction, and their associated topics.

	JSON Payload Type	Direction	Topic
1	IO Board Data	Device -> Cloud	publish_topic
2	Modbus Slave Data	Device -> Cloud	publish_topic
3	Write Acknowledgment	Device -> Cloud	publish_topic
4	SD-Card Fallback Data	Device -> Cloud	publish_topic1
5	Register Write Command	Cloud -> Device	subscriber_topic

1. Modbus Slave Data (Device -> Cloud)

Sent once per configured Modbus slave address. This payload groups all registers belonging to that specific slave into a single JSON object. Field keys are derived from the user-configured "Slave Name" (e.g., "Voltage", "Current").

```
{  
  "DeviceId": "DEV001",  
  "time": "1725600000",  
  "slaveAddress": 1,  
  "Voltage": 230.5000,  
  "Current": 12.3400,  
  "PumpStatus": 1  
}
```

 **wittelB**TM
Our focus is You

(Note: "time" is a Unix timestamp formatted as a string. 32-bit float values are rounded to 4 decimal places before transmission.)

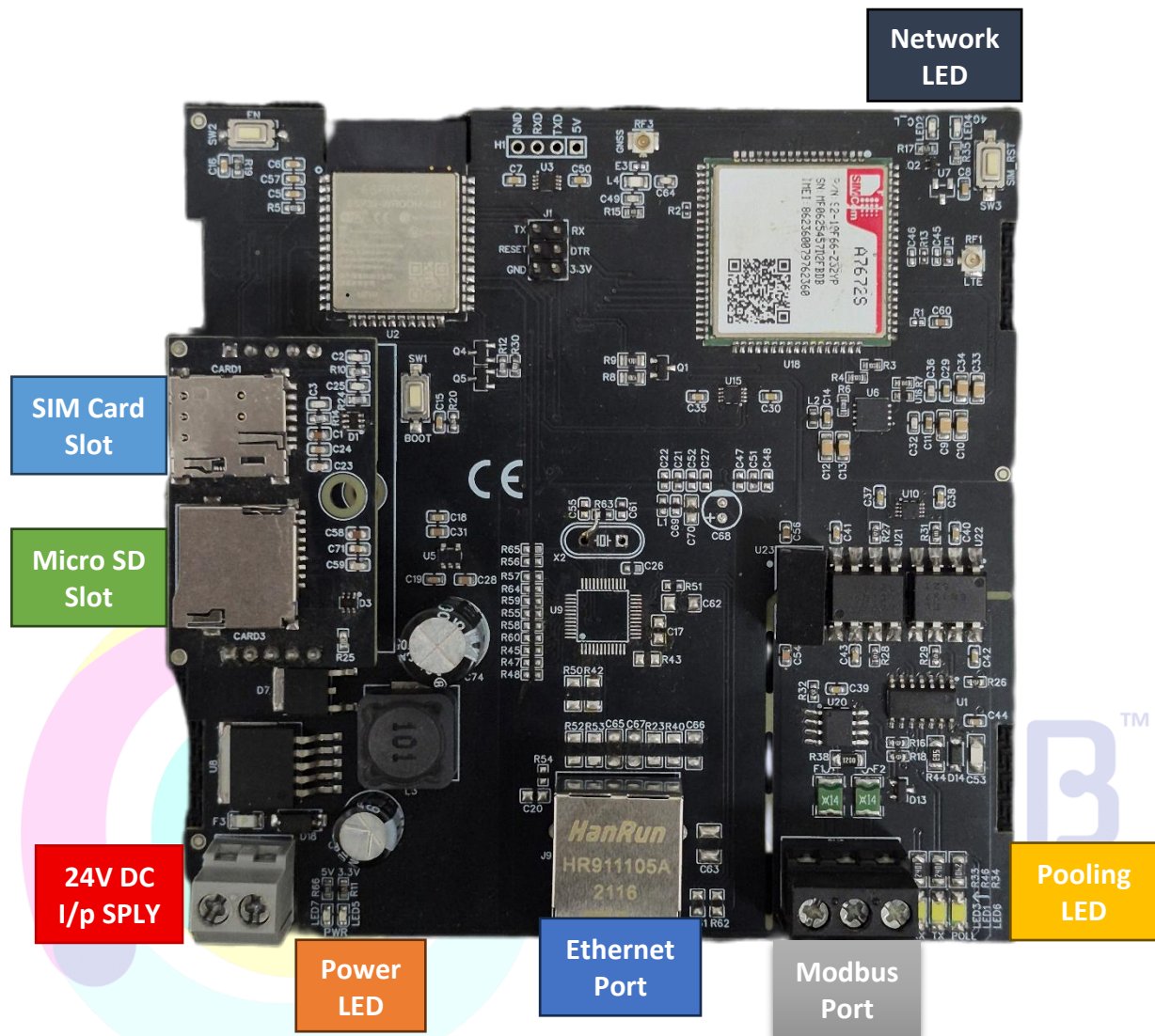
2. Register Write Command (Cloud -> Device)

Sent by the cloud platform to remotely write a value into a holding register. The device will only execute the command if the incoming JSON contains both the register and value keys.

```
{  
  "slaveAddress": 1,  
  "register": 40010,  
  "value": 1  
}
```

(Note: Currently, the device processes this incoming JSON by issuing a Modbus Function Code 6 (Write Single Register) regardless of the target register's actual data type.)

4G Gateway connection Diagram: -



Additional Features: -

Communication ports are isolated
Input power reverse polarity safety
ESD Safety IEC 61000-4-2, $\pm 30\text{KV}$ contact, $\pm 30\text{KV}$ air
EFT IEC 61000-4-4, 50A (5/50ms) 400V isolation.

Contact us: -

Augmatic Technologies Pvt. Ltd.,
Plot no 6, Shah Industrial Estate II,
Kotambi, Vadodara – 391510.

Email – Sales@wittelb.com

Support: support@wittelb.raiseaticket.com / +91 94038 92805